Assignment 1: Algorithmic Thinking with Python

Date : 03/10/2026
Last date of submission : 23/10/2026
Recommended date of submission : 09/10/2026

Total marks : 100

Question No.	Level	Marks	CO Mapped	Bloom's Knowledge Level
Question 1	Easy	7	CO1, CO2	K3 (Apply)
Question 2	Easy	7	CO1, CO2	K3 (Apply)
Question 3	Easy	7	CO1, CO2	K3 (Apply)
Question 4	Easy	7	CO1, CO2	K3 (Apply)
Question 5	Easy	7	CO1, CO2	K3 (Apply)
Question 6	Medium	10	CO1, CO2	K4 (Analyse)
Question 7	Medium	10	CO1, CO2	K4 (Analyse)
Question 8	Medium	10	CO1, CO2	K4 (Analyse)
Question 9	Hard	17.5	CO1, CO2	K5 (Evaluate)
Question 10	Hard	17.5	CO1, CO2	K6 (Create)

1. Instruction to Do the Assignment

1.1 Purpose and Structure of the Assignment

This assignment is both a learning exercise and a test. Students are expected to learn by solving problems carefully and by explaining how the solutions were reasoned. Evaluation is based mainly on the quality of reasoning and not only on the final answer.

The assignment has three sections.

1. Section 1 contains these instructions.
2. Section 2 is the Study Guide. It explains pseudocode, the RAM model and the standard way of writing pseudocode. The Study Guide must be read completely before any question is attempted.
3. Section 3 contains the Assignment Questions. The questions are divided into three levels, namely Easy, Medium and Hard. A fully solved sample problem is given at the beginning of each level. The sample shows how a problem of that level is to be analysed, reasoned out, drafted and written as pseudocode. The sample of a level must be studied before the questions of that level are attempted.

All questions can be answered with basic mathematics and knowledge of conditional statements and looping statements.

1.2 Distribution of Marks

There are 10 questions in total, numbered Question 1 to Question 10. The total is 100 marks.

Easy level: Questions 1 to 5, 7 marks each, 35 marks in total.

Medium level: Questions 6 to 8, 10 marks each, 30 marks in total.

Hard level: Questions 9 and 10, 17.5 marks each, 35 marks in total.

1.3 Marks Required to Pass

Students are not required to attempt all the questions. A score of 65 marks out of 100 is the only requirement to pass the assignment.

The harder the question, the more marks it carries. A hard question is worth 2.5 easy questions, and a medium question is worth about 1.4 easy questions. A student who attempts fewer hard and medium questions must therefore solve more easy questions to reach the required marks. The following combinations would reach 65 marks or more if full marks are obtained.

1. All 5 easy questions and all 3 medium questions give 65 marks.
2. All 3 medium questions and both hard questions give 65 marks.
3. All 5 easy questions and both hard questions give 70 marks.
4. One hard question, all 3 medium questions and 3 easy questions give 68.5 marks.

Marks are awarded for reasoning, and partial marks are possible. Full marks on every question attempted are therefore unlikely, and attempting more questions than the bare minimum is advisable.

1.4 Marks Above 65

Any marks scored above 65 are surplus marks. For example, a score of 80 gives 15 surplus marks. Surplus marks can be used to make up for marks lost in the internal evaluation in the future.

1.5 Clarification of Doubts

Students may approach the teacher with any doubt while solving the questions. Clarification of the meaning of a question, of the Study Guide or of the sample problems may be sought from the teacher at any time. This is permitted and is not considered as seeking help from others. The teacher will not provide the solution to any question, but support will be provided.

1.6 Format of the Answer to Each Question

The answer to every question attempted must be written in the following order.

1. Problem definition, stating the inputs, the outputs and the constraints.
2. Thinking process, explaining in the student's own words how the solution was reasoned.
3. Rough draft of the algorithm in plain English.
4. Pseudocode, following the standard given in the Study Guide.

The solved samples in Section 3 show exactly how each part is to be written.

1.7 Criteria for awarding marks

Every answer will be graded on the following criteria.

1. Thinking process, 50 percent. Hand trials on small inputs, ideas tried and rejected, patterns noticed and the reasons for believing that the approach works must be shown.
2. Clarity of understanding of the problem, 20 percent. What is given, what is asked and what the restrictions are must be stated clearly.

3. Plain English draft and pseudocode standard, 20 percent. The draft must be easy to follow and the pseudocode must follow the formatting standard given in the Study Guide.
4. Correctness of the algorithm, 10 percent.

NOTE : Even if the algorithm is not fully correct or optimal students will still be awarded marks for honest attempts.

1.8 Mode and Date of Submission

Preferably, the answers may be **typed and submitted digitally** as a Word or PDF document. However, if you do not have access to a computing device the answers may also be written very neatly on paper and scanned into a single PDF.

Answers that cannot be read will not be graded.

1.9 Submissions That Will Not Be Accepted

A submission will not be accepted in any of the following cases.

1. The work is copied from any source or from another student.
2. Artificial intelligence tools have been used in any form.
3. The student is unable to explain the algorithm in case of suspicion. Any student may be called to explain any answer in person.
4. Help has been sought from other persons. Approaching the teacher for clarification of doubts, as described in 1.5, is permitted.
5. **The last date for submission is 23 October 2026. Submissions received after this date will not be accepted**

2. Study Guide to Complete the Assignment

2.1 What Is Pseudocode and Why Is It Used

Pseudocode is a way of writing the steps of an algorithm in a structured, English-like form. It resembles a program, but it does not follow the rules of any particular programming language. No compiler exists for it and nothing is run. Its only audience is a human reader.

Pseudocode is used for the following reasons.

1. It allows attention to remain on the logic of the solution instead of the spelling and punctuation rules of a programming language.
2. It is independent of any language. The same pseudocode can later be written in C, Python, Java or any other language.
3. It is easy to read, check and correct. Mistakes in thinking become visible before any program is written.
4. It is a bridge between an idea and a working program. The idea is first described in plain English, then structured as pseudocode, and then converted into code.
5. It shows exactly how the author of an algorithm thinks. This is the reason it is required in this assignment.

2.2 The RAM Model and Pseudocode

To decide whether an algorithm is good or bad, there must be an agreed idea of what a computer can do and how much each action costs. Real computers differ in speed, memory and design. For reasoning about algorithms, a simple imaginary computer called the Random Access Machine, or RAM model, is therefore used.

The RAM model makes the following assumptions.

1. There is one processor. It carries out one instruction at a time, in order. Nothing happens in parallel.
2. Each basic operation takes one unit of time. The basic operations are arithmetic (addition, subtraction, multiplication, integer division and remainder), comparison of two values, assignment of a value to a variable, and reading or writing a variable.
3. Memory is unlimited, and reading any variable costs the same, no matter where it is stored.
4. Each number fits in one memory cell. Numbers too large to store are not considered.
5. Real world effects such as caches, disk speed and network delay are ignored.

The connection with pseudocode is as follows. Every line of pseudocode should describe something that a RAM can do in a small, fixed number of basic operations. A line such as the following is acceptable, because it is one addition and one assignment.

```
sum = sum + digit
```

A line such as "find all the divisors of the number" is not acceptable, because it hides a whole loop. It must be expanded into steps that the RAM can carry out one by one.

The RAM model also allows the amount of work done by an algorithm to be estimated. If a loop runs once for every digit of a number and each pass performs about 4 basic operations, the total work is about 4 times the number of digits. Such estimates are encouraged in the thinking process.

2.3 How to Write Pseudocode

The pseudocode in this assignment must follow the conventions used in the textbook Introduction to Algorithms by Cormen, Leiserson, Rivest and Stein, commonly known as CLRS. These conventions must be followed exactly. Every pseudocode answer must be written inside a code block, or in a clearly separated block in handwritten work, so that the indentation is preserved.

Formatting rules.

1. The pseudocode begins with a header line that gives the name of the procedure followed by its parameters in brackets, for example SUM-OF-DIGITS(n). The name is written in capital letters, with the words joined by hyphens. Where the question uses a capital letter such as N for an input, the corresponding parameter is written in lowercase as n. The inputs and outputs are described in the problem definition and are not written as separate lines. The words BEGIN and END are not used.
2. The lines below the header are numbered 1, 2, 3 and so on, from top to bottom. The header line is not numbered. The line numbers allow any line to be referred to in the explanation.
3. One action is written on each line.
4. Indentation shows the structure of the algorithm. The lines that belong inside an if, elseif, else, while or for are indented by 4 spaces more than the line that controls them. A block ends where the indentation returns to the earlier level. No closing keywords such as END IF or END WHILE, no curly braces, and no words such as then and do are used.
5. The keywords are written in lowercase. They are if, elseif, else, while, for, to, downto, by, return, and, or and not. In printed CLRS pseudocode the keywords appear in bold. In plain text and in handwritten work they are written in lowercase, and in handwritten work they may be underlined.
6. Assignment is written with a single equals sign. The line x = 5 means that the value 5 is stored in x. Equality is tested with a double equals sign, as in n mod 2 == 0. The other comparisons are written with the symbols ≠, <, >, ≤ and ≥.
7. The arithmetic operators are +, −, × and /. The remainder is written with the word mod, for example n mod 10. Integer division, which gives the quotient with the remainder discarded, is written with the floor brackets, for example ⌊n / 10⌋. For instance, 17 mod 5 is 2 and ⌊17 / 5⌋ is 3.
8. A for loop is written as for i = 1 to n. The counter increases by 1 on every pass. The form for i = n downto 1 counts downwards, and the form for i = 1 to n by 2 changes the counter by 2 on every pass. After a for loop ends, the counter keeps its last value, which is the first value beyond the bound.
9. The operators and and or are short circuiting. In the condition x and y, the part y is examined only if x is true. In the condition x or y, the part y is examined only if x is false.
10. A comment begins with // and explains a line or a block. Comments are encouraged where the reason is not obvious.
11. Variable names are written in lowercase and must be meaningful, for example sum, digit and largest. Single letters are used only for loop counters such as i and j, and for the parameters.
12. Parameters are passed by value, which means that the procedure works on its own copy. If the original value of a parameter is needed later in the procedure, it is copied into another variable before it is changed.

13. The result is given with the word return. A procedure may return more than one value, as in return value, power. Where the answer is a word such as YES or NO, it is written in capital letters, as in return YES.
14. Programming language syntax such as semicolons, ++, +=, int, printf or cin is not used.
15. Vague lines such as "do the calculation" or "check everything" are not acceptable. Every line must be a clear, concrete step.

The three structures required. These are shown here without line numbers so that the shape of each structure is clear.

Decision (conditional):

```
if condition
    statements
elseif another condition
    statements
else
    statements
```

Repetition while a condition is true:

```
while condition
    statements
```

Repetition a fixed number of times with a counter:

```
for i = 1 to n
    statements
```

A short complete example that follows all the rules is given below. It finds the larger of two numbers.

```
LARGER-OF-TWO(a, b)
1  if a ≥ b
2      return a
3  else
4      return b
```

2.4 Guidance on the Thinking Process

The thinking process carries the largest share of the marks. The following practices are recommended.

1. Every problem should first be tried by hand on a small input. The steps carried out by hand should be written down, because those steps form the algorithm.
2. After a first idea is obtained, the possible failures should be examined. The smallest input, the largest input, repeated values and zero should be considered.
3. Ideas that were tried and rejected should be recorded together with the reason for rejection.
4. The pseudocode should be run by hand on at least one example to confirm that it gives the right answer.
5. Where possible, the number of basic operations performed by the algorithm should be estimated using the RAM model.

3. Assignment Questions

3.1 Easy Level (Questions 1 to 5, 7 marks each)

Easy questions need one loop or one simple decision structure.

Solved Sample Problem for Easy Level

Problem. Given a positive whole number N, find the sum of its digits. For example, for N = 4062 the answer is 12.

Step 1. Problem definition

Input: one positive whole number N.

Output: one whole number, which is the sum of all the digits of N.

Constraints and restrictions. Only addition, subtraction, multiplication, mod, integer division, comparisons, decisions and loops may be used. The computer cannot be assumed to show the digits one by one. It sees N as one single number.

Samples. N = 4062 gives 4 + 0 + 6 + 2 = 12. N = 7 gives 7.

Step 2. Thinking process

A person who is shown 4062 on paper and asked for the digit sum would simply add 4, 0, 6 and 2. The difficulty is that a computer holds 4062 as a single number and not as four separate digits. The real question is therefore how to get hold of a digit.

The easiest digit to get is the last one. When 4062 is divided by 10, the remainder is 2, and that is exactly the last digit. N mod 10 therefore gives the last digit.

Once the last digit has been used, it must be thrown away so that the next digit becomes the last one. Dividing 4062 by 10 and ignoring the remainder gives 406. □N / 10□ therefore removes the last digit.

The same problem now remains with a smaller number, 406. The same two actions, taking the last digit and removing it, can be repeated until no digits are left. A number with no digits left is 0, so the repetition stops when the number becomes 0.

A place is needed to collect the digits that have been added. A variable called sum is kept and started at 0, because before any digit is looked at, the total is 0.

A test by hand on 4062 gives the following. The number is 4062, so the digit is 2, the sum becomes 2, and the number becomes 406. Next the digit is 6, the sum becomes 8 and the number becomes 40. Next the digit is 0, the sum stays 8 and the number becomes 4. Next the digit is 4, the sum becomes 12 and the number becomes 0. Since the number is 0, the repetition stops. The answer is 12, which is correct.

Possible failures were then considered. When a digit is 0, as in 40, the number is still above 0 after the removal, so the loop correctly continues. A single digit number such as 7 gives one pass: the digit is 7, the sum is 7, the number becomes 0 and the loop stops. The loop always stops, because every division by 10 makes the number strictly smaller until it reaches 0.

The original N is to be kept unchanged, so the work is done on a copy.

Cost in the RAM model. Each pass of the loop does one mod, one addition, one integer division, three assignments and one comparison, which is a small fixed number of operations. The loop runs once per digit, so the total work is proportional to the number of digits of N.

Step 3. Rough draft in plain English

1. Start with a total of 0.

2. Make a copy of N to work on.
3. As long as the copy is greater than 0, repeat the following steps.
4. Take the last digit of the copy, which is the remainder on dividing by 10.
5. Add that digit to the total.
6. Remove the last digit of the copy by dividing it by 10 and discarding the remainder.
7. When the copy reaches 0, the total is the answer.

Step 4. Pseudocode

```
SUM-OF-DIGITS(n)
1  sum = 0
2  remaining = n               // work on a copy so that n is not lost
3  while remaining > 0
4      digit = remaining mod 10
5      sum = sum + digit
6      remaining = □remaining / 10□
7  return sum
```

Question 1 (7 marks)

Given a positive whole number N, find the largest digit that appears in N. For example, for N = 52917 the answer is 9, and for N = 3 the answer is 3.

The thinking process must state the value used as the starting guess for the largest digit, and must explain why that starting value cannot cause a wrong answer.

Question 2 (7 marks)

Given a positive whole number N, find the sum of all the whole numbers below N that are multiples of 3 or multiples of 5. For example, for N = 10 the numbers are 3, 5, 6 and 9, and the answer is 23.

A number such as 15 is a multiple of both 3 and 5 and must be counted only once. The thinking process must explain clearly how the algorithm ensures this.

Question 3 (7 marks)

Given three different numbers a, b and c, find the middle number, that is, the one that is neither the smallest nor the largest. For example, for the numbers 7, 2 and 9 the answer is 7.

Only comparisons and decisions may be used. The numbers must not be arranged in order by any method. The thinking process must list every possible ordering of the three numbers and show that the algorithm handles each one.

Question 4 (7 marks)

A year is a leap year under the following rules. A year divisible by 4 is a leap year, except that a year divisible by 100 is not a leap year, unless it is also divisible by 400, in which case it is a leap year. Given a year Y, decide whether it is a leap year. The output is the word LEAP or NOT LEAP.

The thinking process must include at least five test years considered to be the most difficult for an algorithm. The reason each is difficult must be explained, and the algorithm must be shown to give the right answer for each.

Question 5 (7 marks)

Given a positive whole number N, find the smallest power of 2 that is greater than or equal to N, and the power to which 2 is raised. For example, for N = 37 the answer is 64, which is 2

raised to 6. For N = 64 the answer is 64 itself, and for N = 1 the answer is 1, which is 2 raised to 0.

The thinking process must explain how the algorithm knows when to stop, and why it cannot run forever.

3.2 Medium Level (Questions 6 to 8, 10 marks each)

Medium questions need a loop combined with a decision, or the tracking of more than one piece of information while looping, or the improvement of a first idea.

Solved Sample Problem for Medium Level

Problem. A positive whole number N is called a perfect number if it is equal to the sum of its proper divisors. The proper divisors of N are all the whole numbers that divide N exactly, except N itself. For example, 6 is perfect because 1 + 2 + 3 = 6. Decide whether a given N is a perfect number. The output is YES or NO.

Step 1. Problem definition

Input: one positive whole number N.

Output: the word YES if N is perfect, otherwise NO.

Constraints and restrictions. Only loops, decisions, mod, integer division and basic arithmetic may be used. A number d is a divisor of N if N mod d is 0. N itself is not counted. The number 1 is a proper divisor of every N greater than 1.

Samples. N = 6 gives YES. N = 28 gives YES because 1 + 2 + 4 + 7 + 14 = 28. N = 12 gives NO because 1 + 2 + 3 + 4 + 6 = 16, which is not 12.

Step 2. Thinking process

The first point to settle is what "divides exactly" means for a computer. A number d divides N exactly when the remainder on dividing N by d is 0, so the test is that N mod d equals 0.

Done by hand for N = 28, the numbers 1, 2, 3 and so on would be taken one after another and each would be tested for whether it divides 28. The ones that do, namely 1, 2, 4, 7 and 14, would be written down and added to give 28. The total would then be compared with 28.

The plan therefore has three parts. Every candidate number is tried, each is tested, and a running total of the ones that pass is kept. At the end, the total is compared with N.

The candidates to be tried are every number from 1 up to N minus 1, since N itself is not allowed. This is certainly correct, so it is written first and improved afterwards.

Fewer numbers may be tried. The second largest divisor of N cannot be bigger than half of N, because dividing N by something larger than half of N gives a quotient less than 2, and the only such case is the divisor N itself. Only the numbers from 1 up to □N / 2□ therefore need to be tried. This halves the work and still gives the correct answer.

Edge cases were checked. For N = 1, □N / 2□ is 0, so the loop does not run at all, the total stays 0, and 0 is not 1, so the answer is NO. This is correct, because 1 has no proper divisors. For N = 2, only 1 is tried, the total is 1, which is not 2, so the answer is NO. This is also correct.

A test with N = 12 follows. The numbers 1 to 6 are tried. The divisors are 1, 2, 3, 4 and 6, and the total is 16. Since 16 is not 12, the answer is NO. This is correct.

The total must be set to 0 before the loop starts, otherwise it would carry a wrong starting value.

Cost in the RAM model. The loop runs about N divided by 2 times, with a mod, a comparison and possibly an addition each time. The work therefore grows in proportion to N.

Step 3. Rough draft in plain English

1. Start the total of divisors at 0.
2. For every number d from 1 up to half of N (ignoring any fraction), do the following.
3. If d divides N exactly, add d to the total.
4. After all numbers have been tried, compare the total with N.
5. If they are equal, the answer is YES. Otherwise the answer is NO.

Step 4. Pseudocode

```
IS-PERFECT(n)
1  total = 0
2  for d = 1 to ⌊n / 2⌋
3      if n mod d == 0
4          total = total + d
5  if total == n
6      return YES
7  else
8      return NO
```

Question 6 (10 marks)

Given a positive whole number N, decide whether it is a palindrome, that is, whether it reads the same from left to right and from right to left. The output is YES or NO. For example, 12321 gives YES and 1230 gives NO.

N must not be converted into text or broken into separate digit positions by any method other than arithmetic. The thinking process must explain what goes wrong with numbers that end in 0, such as 120 or 1001, and must show how the algorithm deals with them.

Question 7 (10 marks)

Given two positive whole numbers a and b, find their greatest common divisor, that is, the largest whole number that divides both of them exactly. For example, for 12 and 18 the answer is 6.

The solution must be developed in two stages. First, a simple algorithm that tries candidate numbers one by one must be designed. Then a faster method must be thought out and designed. For a = 1071 and b = 462, the number of mod operations performed by each of the two algorithms must be counted and compared.

A thinking prompt, not a solution. If a number d divides both a and b, what must be true of d with respect to the difference of a and b, and what does this suggest about replacing a pair of numbers by a smaller pair that has the same common divisors?

Question 8 (10 marks)

A list of positive whole numbers is given one number at a time. The list ends with the number 0, which is not part of the list. Find the second largest distinct value in the list. If the list has fewer than two distinct values, the output is NONE. For example, the list 4, 9, 9, 2, 0 gives 4, and the list 5, 5, 5, 0 gives NONE.

The whole list must not be stored. Each number can be looked at only once, as it arrives. The thinking process must explain what information has to be remembered at every step, and what is done in each of three cases when a new number arrives: it is larger than the largest so far, it is equal to the largest so far, and it lies between the second largest and the largest.

3.3 Hard Level (Questions 9 and 10, 17.5 marks each)

Hard questions require insight and not only effort. A direct method may exist but be far too slow, so a smarter way must be discovered. Alternatively, a longer solution made of several carefully coordinated parts must be designed.

Solved Sample Problem for Hard Level

Problem. Given a positive whole number N, find how many zeros appear at the end of N factorial. N factorial, written N!, is the product 1 × 2 × 3 × ... × N. For example, 5! = 120 has 1 zero at the end and 10! = 3628800 has 2 zeros at the end. N can be as large as 1000000000.

Step 1. Problem definition

Input: one positive whole number N, which may be as large as 1000000000.

Output: one whole number, the count of zeros at the very end of N!.

Constraints and restrictions. Only basic arithmetic, mod, integer division, loops and decisions may be used. Because N can be huge, N! cannot be computed in practice. Already 20! is more than two quintillion, and 100! has 158 digits. The algorithm must therefore find the count of zeros without ever building N!.

Samples. N = 5 gives 1. N = 10 gives 2. N = 4 gives 0, because 4! = 24.

Step 2. Thinking process

The first idea is to compute N! and then count the zeros at the end with the digit method: while the last digit is 0, count it and remove it by dividing by 10. This works for small N. But N can be a billion, and N! for that is unimaginably large. The first idea is correct but unusable. The zeros must be found without building the number.

A zero at the end comes from a multiple of 10. The question is therefore how many times 10 divides into N!. Now 10 is 2 times 5. Every zero needs one factor 2 and one factor 5 hiding somewhere in the product 1 × 2 × ... × N. The number of zeros is the number of such pairs that can be formed.

A small case, 10!, is examined. The factors of 2 hide in 2, 4, 6, 8 and 10. Counting them, 2 gives one, 4 gives two, 6 gives one, 8 gives three and 10 gives one, a total of 8. The factors of 5 hide in 5 and 10, which gives 2 of them. Only 2 pairs can be formed, and 10! indeed ends in 2 zeros.

It is noticed that factors of 2 are far more plentiful than factors of 5, because every second number brings a 2 while only every fifth number brings a 5. The 5s will always run out first, so the number of zeros equals the number of factors of 5 in the product. This is the key insight. The problem is now reduced to counting the factors of 5.

Every multiple of 5 contributes at least one factor of 5. The number of multiples of 5 up to N is □N / 5□. For N = 10 this gives 2, which matches.

A test with a larger N shows a problem. For N = 30, the multiples of 5 are 5, 10, 15, 20, 25 and 30, which is 6. But 25 is 5 times 5 and brings two factors of 5, not one. The real count is therefore 7. The extra one comes from the multiples of 25. Likewise a multiple of 125 brings a third factor, and so on.

The count is therefore as follows. The multiples of 5 are counted, which is □N / 5□. The multiples of 25 are added, which is □N / 25□, since those numbers each have one more factor. The multiples of 125 are added, which is □N / 125□, and so on, until the power of 5 is bigger than N. For N = 30 this gives 6 + 1 = 7. For N = 125 it gives 25 + 5 + 1 = 31.

Writing out the powers 5, 25, 125 explicitly is awkward. It is noticed, however, that □N / 25□ is the same as □□N / 5□ / 5□. Instead of dividing N by larger and larger powers, it is possible

to divide by 5 again and again. Start with q = □N / 5□ and add it. Then replace q by □q / 5□ and add it. Continue until q is 0.

A test with N = 125 follows. q = 25, so the count is 25. Then q = 5, so the count is 30. Then q = 1, so the count is 31. Then q = 0, so the repetition stops. The answer is 31, which agrees with the earlier calculation. A test with N = 4 gives q = 0 immediately, so the answer is 0, which agrees with 24. A test with N = 5 gives q = 1, so the count is 1, which agrees with 120.

The loop always stops, because q becomes smaller on every step and reaches 0.

Cost in the RAM model. Each pass does one addition, one division and a comparison. For N = 1000000000, the loop needs only about 13 passes. This is a huge saving compared with computing N!.

Step 3. Rough draft in plain English

1. Start with a count of zeros equal to 0.
2. Divide N by 5, discarding the remainder, and call the result q.
3. As long as q is greater than 0, repeat the following steps.
4. Add q to the count.
5. Replace q by q divided by 5, discarding the remainder.
6. When q reaches 0, the count is the number of zeros at the end of N!.

Step 4. Pseudocode

```
TRAILING-ZEROS-OF-FACTORIAL(n)
1  zeros = 0
2  q = □n / 5□                  // multiples of 5 up to n
3  while q > 0
4       zeros = zeros + q
5       q = □q / 5□             // the next round counts multiples of 25,
then 125, and so on
6  return zeros
```

Question 9 (17.5 marks)

Given a whole number N that is at least 2, find the largest prime factor of N. A prime factor is a prime number that divides N exactly. For example, for N = 13195 the prime factors are 5, 7, 13 and 29, so the answer is 29. For N = 17 the answer is 17.

N can be as large as 1000000000000. Trying every number from 1 to N is too slow. The thinking process must first describe the direct method and estimate its work for the largest N. It must then show how the work can be reduced greatly. It is useful to examine by hand what happens to the number when it is repeatedly divided by its smallest factor, and to decide when the search can safely stop. The largest number of passes that the final algorithm may need must be estimated using the RAM model.

Question 10 (17.5 marks)

There are N people standing in a circle, numbered 1 to N in order. Starting with person 1, the circle is traversed and the people are counted alternately as skipped and eliminated. Person 1 is skipped, person 2 is eliminated, person 3 is skipped, person 4 is eliminated, and so on. The traversal continues around the circle, ignoring those already eliminated, until only one person is left. Find the number of the person who survives.

For example, for N = 5, the eliminated people in order are 2, 4, 1 and 5, so person 3 survives. For N = 7 the survivor is person 7.

N can be as large as 1000000000, so simulation of the whole circle is not practical. The thinking process must contain a hand simulation for every N from 1 to at least 10, laid out so that a pattern can be seen. The pattern noticed must be described, and an explanation of why it must hold must be given in the student's own words. Only then must the algorithm be designed. The algorithm must be tested on at least three values of N that were not used for finding the pattern.